Transformer Concepts — Deep Dive

We'll use the same running example:

```
Source: "I am a student"  →  Target: "je suis un étudiant"
d_model = 512, n_heads = 8
```

1. Why Not RNN? The Parallelism Problem

How RNN processes a sentence (Sequential)

RNN reads words **one by one**, left to right. Each word must wait for the previous word to finish.

```
"I am a student"

Step 1: Process "I"       → hidden_state_1
Step 2: Process "am"      → hidden_state_2  (needs hidden_state_1 — must WAIT)
Step 3: Process "a"       → hidden_state_3  (needs hidden_state_2 — must WAIT)
Step 4: Process "student" → hidden_state_4  (needs hidden_segment_3 — must WAIT)

Total: 4 sequential steps. Cannot parallelize.
```

```
Time →
  ┌───┐
  │ I │──→ h1
  └───┘     │
            ▼
          ┌────┐
          │ am │──→ h2
          └────┘     │
                     ▼
                   ┌───┐
                   │ a │──→ h3
                   └───┘     │
                             ▼
                         ┌─────────┐
                         │ student │──→ h4
                         └─────────┘

Each step DEPENDS on the previous one → slow, sequential
```

Problems with RNN:

- **Slow:** Can't process words in parallel — must go one by one

- **Forgets:** By the time it reaches "student", it may have forgotten details about "I" (vanishing gradient problem)
- **Long distance:** Hard to connect words far apart (e.g., "The cat, which sat on the mat, **was** happy" — "was" needs to connect to "cat" many steps back)

How Transformer processes a sentence (Parallel)

Transformer processes **all words at the same time** using attention.

```
"I am a student"

Step 1: Process "I", "am", "a", "student" → ALL AT ONCE, IN PARALLEL

Total: 1 step. Fully parallelized.
```

```
Time →
  ┌───┐   ┌────┐   ┌───┐   ┌─────────┐
  │ I │   │ am │   │ a │   │ student │    ← all processed simultaneously
  └───┘   └────┘   └───┘   └─────────┘
    │       │       │         │
    ▼       ▼       ▼         ▼
  ┌──────────────────────────────────┐
  │     Self-Attention Layer         │  ← every word looks at every other word
  └──────────────────────────────────┘
    │       │       │         │
    ▼       ▼       ▼         ▼
   h1      h2      h3        h4       ← all outputs computed at once
```

Why parallel?

- No word depends on the previous word's output
- Every word directly attends to every other word through matrix multiplication
- Matrix multiplication is what GPUs are built for → massive speedup

Comparison:

Feature	RNN	Transformer
Processing	Sequential (word by word)	Parallel (all words at once)
Speed	Slow (N steps for N words)	Fast (1 step for N words)
Long-range context	Weak (forgets over time)	Strong (direct attention)
Training	Hard to parallelize	Fully parallelizable on GPU
Position awareness	Built-in (sequential)	Needs positional encoding

---

2. What is Attention?

The Core Idea

Attention answers one question: **"When processing this word, how much should I focus on each other word?"**

Think of it like reading a sentence — when you read "am", your brain automatically focuses more on "I" (the subject) than on "a" (an article). Attention does the same thing mathematically.

The Q, K, V Framework

Every word gets transformed into three vectors:

Vector	Full Name	Role	Analogy
Q (Query)	Query	"What am I looking for?"	A search query you type
K (Key)	Key	"What do I contain?"	A title/tag on a document
V (Value)	Value	"Here is my actual information"	The content of that document

Analogy — Library Search:

```
You want to know: "Who is the subject of 'am'?"  ← this is the Query (from "am")

Each word in the sentence has a label:
  "I"       → Key: "I'm a pronoun/subject"        Value: [information about "I"]
  "am"      → Key: "I'm a verb"                   Value: [information about "am"]
  "a"       → Key: "I'm an article"               Value: [information about "a"]
  "student" → Key: "I'm a noun/object"            Value: [information about "student"]

You compare your Query with each Key → get relevance scores:
  Q("am") × K("I")       = 0.85  ← very relevant! (subject-verb)
  Q("am") × K("am")      = 0.40
  Q("am") × K("a")       = 0.05  ← not relevant
  Q("am") × K("student") = 0.60  ← somewhat relevant (verb-object)

Then you grab the Values, weighted by these scores:
  output = 0.85 × V("I") + 0.40 × V("am") + 0.05 × V("a") + 0.60 × V("student")

Result: "am" now carries mostly information from "I" and "student"
```

These weights are not manually assigned — they are learned by the model during training through backpropagation.

Here's how it actually works:

Step 1: Each word starts as an embedding vector (512 numbers)

"am" = [0.2, 0.8, -0.1, 0.5, ...] (512 dims) "I" = [0.3, 0.7, 0.0, -0.2, ...] (512 dims)

Step 2: These embeddings are multiplied by learned weight matrices (W_Q, W_K) to create Q and K

Q("am") = "am" embedding × W_Q = [0.4, 0.9, -0.3, ...] (64 dims) K("I") = "I" embedding × W_K = [0.5, 0.8, 0.1, ...] (64 dims)

W_Q and W_K are the model's weights — random at first, improved during training.

Step 3: The score is just a dot product (multiply matching positions and sum)

Q("am") × K("I") = 0.4×0.5 + 0.9×0.8 + (-0.3)×0.1 + ... = 0.20 + 0.72 + (-0.03) + ... = some number (e.g., 3.4)

Step 4: After scaling (÷ √d_k) and softmax, we get the final weights

Raw scores: [3.4, 1.8, 0.1, 2.5] ← dot products Scaled: [0.43, 0.23, 0.01, 0.31] ← ÷ √64 After softmax: [0.35, 0.22, 0.15, 0.28] ← probabilities (sum to 1)

So the 0.85 in my example was a simplified illustration. In reality:

Before training — W_Q and W_K are random → attention scores are random → model predicts garbage

During training — backpropagation adjusts W_Q and W_K so that Q("am") and K("I") produce a high dot product (because "I" is relevant to "am"), and Q("am") and K("a") produce a low dot product (because "a" is not relevant)

After training — the model has learned weight matrices that produce meaningful attention patterns

The model discovers that subject-verb pairs should have high scores — nobody tells it that. It learns this from seeing thousands of examples and reducing the loss.

Attention Formula

```
Attention(Q, K, V) = softmax(Q × Kᵀ / √d_k) × V
```

Let's break this down step by step:

Step 1: Q × K^T (Dot Product — compute similarity scores)

```
Each Q and K is a vector of size d_k (e.g., 64)

Q("am")  = [0.2, 0.8, -0.1, ...]    (64 dims)
K("I")   = [0.3, 0.7, 0.0, ...]     (64 dims)

Q × Kᵀ = dot product = 0.2×0.3 + 0.8×0.7 + (-0.1)×0.0 + ... = score

Higher score = more similar/relevant
```

For all words at once (matrix multiplication):

```
          K("I")  K("am")  K("a")  K("student")
Q("I")   [ 2.1    0.5     0.3      1.2  ]
Q("am")  [ 3.4    1.8     0.1      2.5  ]    ← raw attention scores
Q("a")   [ 0.2    0.1     0.8      1.5  ]
Q("stud")[ 1.9    2.3     1.1      0.7  ]
```

Step 2: ÷ √d_k (Scaling)

```
d_k = 64  →  √64 = 8

Why scale? Without scaling, dot products get very large for high dimensions.
```

```
Large values → softmax becomes extremely peaked (almost one-hot) → gradients vanish.
Dividing by √d_k keeps values in a reasonable range.

Before scaling: [3.4, 1.8, 0.1, 2.5]
After scaling:  [0.425, 0.225, 0.0125, 0.3125]
```

Step 3: Softmax (Convert scores to probabilities)

```
softmax([0.425, 0.225, 0.0125, 0.3125])

= [e^0.425, e^0.225, e^0.0125, e^0.3125] / sum_of_all

= [0.35, 0.22, 0.15, 0.28]    ← attention weights (sum to 1.0)

Now we know: "am" should pay 35% attention to "I", 22% to "am", 15% to "a", 28% to "student"
```

Step 4: × V (Weighted sum of values)

```
output("am") = 0.35 × V("I") + 0.22 × V("am") + 0.15 × V("a") + 0.28 × V("student")

Result: a new vector for "am" that contains context from all relevant words
```

Full picture as matrices:

```
Input: 4 words, each 64 dims

Q = (4, 64)     K = (4, 64)     V = (4, 64)

Step 1: Q × Kᵀ           = (4, 64) × (64, 4) = (4, 4)     ← score matrix
Step 2: ÷ √64            = (4, 4)                          ← scaled scores
Step 3: softmax           = (4, 4)                          ← attention weights
Step 4: × V              = (4, 4) × (4, 64)  = (4, 64)     ← context-rich output

This is ALL matrix multiplication → runs in parallel on GPU!
```

---

3. Self-Attention vs Cross-Attention

Self-Attention (used in Encoder & Decoder)

"Self" means Q, K, V all come from the **same** sentence.

```
Encoder Self-Attention:
  Input: "I am a student"
  Q = from "I am a student"
  K = from "I am a student"     ← same sentence!
  V = from "I am a student"
```

```
  Purpose: each source word understands its relationship with every other source word

  Example — what "student" learns:
    "student" attends to "I"  → knows who the student is
    "student" attends to "am" → knows it's a present state
    "student" attends to "a"  → knows it's singular
```

```
Decoder Masked Self-Attention:
  Input: " je suis un"
  Q = from " je suis un"
  K = from " je suis un"   ← same sentence!
  V = from " je suis un"

  Purpose: each target word understands its relationship with previous target words
  (future words are MASKED — see causal mask in process.md Step 7)
```

Cross-Attention (used in Decoder only)

"Cross" means Q comes from one sentence, K and V come from a **different** sentence.

```
Decoder Cross-Attention:
  Q = from decoder (target):  " je suis un"     ← "what am I looking for?"
  K = from encoder (source):  "I am a student"        ← "what does the source contain?"
  V = from encoder (source):  "I am a student"        ← "give me the source information"

  Purpose: each target word finds the most relevant source word to translate from
```

Example:

```
When generating "suis":
  Q("suis") asks: "which source word should I translate from?"

  Attention scores:
    "suis" → "I"       = 0.10
    "suis" → "am"      = 0.65  ← highest! "suis" = "am" in French
    "suis" → "a"       = 0.05
    "suis" → "student" = 0.20

  Result: "suis" pulls most of its information from "am"
```

Summary:

Type	Q from	K, V from	Where used	Purpose
Self-Attention	Same input	Same input	Encoder, Decoder	Understand relationships within a sentence
Cross-Attention	Target	Source (memory)	Decoder only	Connect target words to source words

---

4. Multi-Head Attention

Why multiple heads?

A single attention can only focus on one type of relationship at a time. But language has many relationships happening simultaneously:

```
"I am a student"

  Grammatical:  "am" relates to "I" (subject-verb agreement)
  Semantic:     "student" relates to "I" (who is the student)
  Positional:   "a" relates to "student" (article before noun)
  Dependency:   "am" relates to "student" (linking verb to predicate)
```

One attention head might focus on grammar, another on meaning, another on position. **Multiple heads capture all these relationships in parallel.**

How it works

Instead of one big attention with `d_model=512`, we split into `n_heads=8` smaller attentions, each with `d_k = d_model / n_heads = 512 / 8 = 64`.

Step 1: Project into multiple heads

```
Input X: (4 words, 512 dims)

For each head, we have separate learned weight matrices:
  W_Q_1, W_K_1, W_V_1  → Head 1 (512 → 64)
  W_Q_2, W_K_2, W_V_2  → Head 2 (512 → 64)
  ...
  W_Q_8, W_K_8, W_V_8  → Head 8 (512 → 64)

Head 1: Q1 = X × W_Q_1,  K1 = X × W_K_1,  V1 = X × W_V_1   → each (4, 64)
Head 2: Q2 = X × W_Q_2,  K2 = X × W_K_2,  V2 = X × W_V_2   → each (4, 64)
...
Head 8: Q8 = X × W_Q_8,  K8 = X × W_K_8,  V8 = X × W_V_8   → each (4, 64)
```

Step 2: Run attention on each head IN PARALLEL

```
Head 1: Attention(Q1, K1, V1) → output_1 (4, 64)   — maybe learns grammar
Head 2: Attention(Q2, K2, V2) → output_2 (4, 64)   — maybe learns meaning
Head 3: Attention(Q3, K3, V3) → output_3 (4, 64)   — maybe learns position
...
Head 8: Attention(Q8, K8, V8) → output_8 (4, 64)   — maybe learns something else

All 8 heads run at the SAME TIME (parallel on GPU)
```

Step 3: Concatenate all heads

```
concat = [output_1 | output_2 | ... | output_8]

Shape: (4, 64) × 8 heads = (4, 512)   ← back to original d_model size
```

Step 4: Final linear projection

```
final_output = concat × W_O     where W_O is (512, 512)

Shape: (4, 512) → (4, 512)

This mixes information from all heads into the final representation.
```

Visual summary:

```
Input (4, 512)
    │
    ├──→ Head 1 (4, 64) ──→ Attention ──→ (4, 64) ┐
    ├──→ Head 2 (4, 64) ──→ Attention ──→ (4, 64) ┤
    ├──→ Head 3 (4, 64) ──→ Attention ──→ (4, 64) ┤
    ├──→ Head 4 (4, 64) ──→ Attention ──→ (4, 64) ┤   ALL IN PARALLEL
    ├──→ Head 5 (4, 64) ──→ Attention ──→ (4, 64) ┤
    ├──→ Head 6 (4, 64) ──→ Attention ──→ (4, 64) ┤
    ├──→ Head 7 (4, 64) ──→ Attention ──→ (4, 64) ┤
    └──→ Head 8 (4, 64) ──→ Attention ──→ (4, 64) ┘
                                                  │
                                            Concatenate
                                                  │
                                             (4, 512)
                                                  │
                                       Linear Projection (W_O)
                                                  │
                                       Output (4, 512)
```

What each head might learn:

```
Head 1: "am" pays attention to "I"        → subject-verb relationship
Head 2: "student" pays attention to "a"   → article-noun relationship
Head 3: "am" pays attention to "student"  → verb-predicate relationship
Head 4: each word pays attention to nearby words → local context
Head 5: "I" pays attention to "student"   → long-range coreference
...
(the model learns what each head should focus on during training)
```

---

5. Frequently Asked Questions

Q: How does the Transformer know word order without being sequential?

A: Positional Encoding. Since all words are processed in parallel, the model has no idea that "I" comes before "am". Positional encoding adds a unique pattern to each position:

```
Position 0: add [0.00, 1.00, 0.00, 1.00, ...]
Position 1: add [0.84, 0.54, 0.01, 1.00, ...]
Position 2: add [0.91, -0.42, 0.02, 0.99, ...]

These patterns are unique for each position, so the model can distinguish
"I am a student" from "student a am I"
```

Q: Why is softmax used in attention?

A: Two reasons:

1. Converts raw scores into **probabilities** (0 to 1, sum to 1) — so we get a proper weighted average
2. **Amplifies differences** — high scores get higher, low scores get pushed toward 0

```
Raw scores:    [3.4, 1.8, 0.1, 2.5]
After softmax: [0.45, 0.18, 0.03, 0.34]

Notice: 3.4 → 0.45 (dominant), 0.1 → 0.03 (nearly ignored)
```

Q: Why divide by √d_k in attention?

A: Without scaling, dot products grow large as dimension increases. Large values make softmax output nearly one-hot (all weight on one word), which causes:

- Vanishing gradients (softmax saturates)
- Model can't learn nuanced attention patterns

```
Without scaling (d_k=64):
  scores: [54.2, 12.1, 3.4, 41.8]
  softmax: [0.99, 0.00, 0.00, 0.01]  ← almost one-hot, gradient ≈ 0

With scaling (÷ √64 = ÷ 8):
  scores: [6.78, 1.51, 0.43, 5.23]
  softmax: [0.45, 0.18, 0.03, 0.34]  ← smooth distribution, healthy gradients
```

Q: What is the difference between Attention and Self-Attention?

A: "Attention" is the general mechanism (Q, K, V can come from anywhere). "Self-Attention" is a specific case where Q, K, V all come from the **same** input.

```
Attention:       Q from sentence A, K & V from sentence B  (cross-attention)
Self-Attention:  Q, K, V all from sentence A               (looking at itself)
```

Q: Why do we need both Self-Attention AND Cross-Attention in the decoder?

A:

- **Masked Self-Attention:** "What have I generated so far?" — understands the target sentence being built
- **Cross-Attention:** "What part of the source should I translate next?" — connects to the source sentence

```
Generating "suis" in "je suis un étudiant":

  Masked Self-Attention:
    "suis" looks at "" and "je" → understands "I've started with 'je'"

  Cross-Attention:
    "suis" looks at "I am a student" → finds "am" is most relevant → translates "am" → "suis"

Both are needed — one for target context, one for source context.
```

Q: Why is the decoder's self-attention "masked"?

A: During training, we feed the entire target sentence at once (for parallel speed). But the model should learn to predict left-to-right. Without masking, when predicting "suis", the model could just look ahead and see "suis" — that's cheating!

```
Without mask: "suis" can see [, je, suis, un, étudiant] → just copies the answer
With mask:    "suis" can see [, je]                     → must actually learn to predict
```

Q: What are Residual Connections (Add & Norm) and why are they needed?

A: After each sub-layer (attention or feed-forward), we do:

```
output = LayerNorm(input + sublayer_output)
```

- **Add (Residual):** `input + sublayer_output` — the original information is preserved. If the sub-layer learns nothing useful, the data just passes through unchanged. This prevents the "degradation problem" in deep networks.
- **LayerNorm:** Normalizes values to have mean=0, std=1. Keeps numbers in a stable range so training doesn't diverge.

```
Without residual: information must pass through 6 layers → original signal gets distorted
With residual:    original signal has a "shortcut" path → always preserved
```

```
input ──────────────────────────┐
  │                             │ (shortcut/skip connection)
  ▼                             │
Attention/FFN                   │
  │                             │
  ▼                             │
  + ◄───────────────────────────┘  (Add)
  │
  ▼
LayerNorm                         (Normalize)
  │
  ▼
output
```

Q: What does the Feed-Forward Network (FFN) do?

A: After attention gathers context, FFN processes each position independently to add non-linear transformations. It's a simple 2-layer network:

```
FFN(x) = ReLU(x × W1 + b1) × W2 + b2

Dimensions:
  x:  (512)  →  W1: (512, 2048)  →  ReLU  →  W2: (2048, 512)  →  output: (512)

The inner dimension (2048 = 4 × d_model) is larger — this "expands" the representation,
lets the model learn complex patterns, then "compresses" back.
```

Why needed? Attention only does weighted averaging (linear). FFN adds non-linearity (ReLU) so the model can learn complex, non-linear patterns.

Q: What is Teacher Forcing?

A: During training, instead of feeding the decoder its own (possibly wrong) predictions, we feed it the **correct** previous words.

```
Without teacher forcing (autoregressive):
  Step 1: []           → predicts "le" (WRONG, should be "je")
  Step 2: [, le]       → predicts "est" (wrong builds on wrong)
  Step 3: [, le, est]  → completely off track

With teacher forcing:
  We always feed the correct target: [, je, suis, un]
  Step 1: given []              → should predict "je"
  Step 2: given [, je]          → should predict "suis"
  Step 3: given [, je, suis]    → should predict "un"

  Even if the model predicts wrong, next step gets the correct input → stable training
```

Q: What is Label Smoothing?

A: Instead of telling the model "the answer is 100% this word", we say "the answer is 90% this word and 10% spread across all other words."

```
Without label smoothing:
  Target: [0, 0, 0, 1.0, 0, 0, ...]     ← 100% on correct word

With label smoothing (0.1):
  Target: [0.001, 0.001, 0.001, 0.9, 0.001, 0.001, ...]  ← 90% correct, 10% spread

Why? Prevents the model from becoming overconfident. An overconfident model:
  - Outputs extreme logits → large gradients → unstable training
  - Doesn't generalize well to new sentences
```

Q: What is Gradient Clipping?

A: Caps the size of gradients during backpropagation to prevent "exploding gradients."

```
Without clipping:
  Some gradient = 1500.0  → weight update is HUGE → model parameters go crazy → loss = NaN

With clipping (max_norm=1.0):
  If total gradient norm > 1.0, scale ALL gradients down proportionally
  gradient 1500.0 → scaled to ~0.5  → stable, controlled update
```

Q: Why does the Encoder run only ONCE during inference but the Decoder runs in a loop?

A: The source sentence doesn't change — it's always "I am a student". So we encode it once and reuse that memory. But the target grows with each new word, so the decoder must run again each time.

```
Encoder: "I am a student" → memory (computed ONCE, reused every step)

Decoder:
  Step 1: []                          + memory → "je"
  Step 2: [, je]                      + memory → "suis"
  Step 3: [, je, suis]               + memory → "un"
  Step 4: [, je, suis, un]           + memory → "étudiant"
  Step 5: [, je, suis, un, étudiant] + memory →  (STOP)

Same memory every time, but decoder input keeps growing.
```

Q: What is Greedy Decoding vs Beam Search?

A:

```
Greedy Decoding:
  At each step, pick the word with the HIGHEST probability.
  Fast but may miss better overall translations.

  Step 1: P("je")=0.7, P("le")=0.2, P("la")=0.1  → pick "je"
  Step 2: P("suis")=0.8, P("ai")=0.15            → pick "suis"
  Final: "je suis" (may not be the best overall sequence)

Beam Search (beam_size=3):
  Keep the TOP 3 candidates at each step, explore all paths.
  Slower but finds better translations.

  Step 1: keep ["je" (0.7), "le" (0.2), "la" (0.1)]
  Step 2: expand each:
    "je suis" (0.7 × 0.8 = 0.56)
    "je ai"   (0.7 × 0.15 = 0.105)
    "le est"  (0.2 × 0.6 = 0.12)
    ... keep top 3 ...
  Final: picks the sequence with highest total probability
```

---

6. Transformer Rules & Constraints

Hard Rules (break these → model crashes or won't work)

Rule 1: d_model must stay the same everywhere

```
Embedding output     = 512 ─┐
Positional encoding  = 512  │
Encoder input        = 512  │
Encoder output       = 512  ├── ALL must be the SAME
Decoder input        = 512  │
Decoder output       = 512  │
Cross-attention K,V  = 512 ─┘

Why? Because of residual connections:
  output = input + sublayer_output

  input = (4, 512)  +  sublayer_output = (4, 512)  → works ✓
  input = (4, 512)  +  sublayer_output = (4, 256)  → can't add → crash ✗
```

Rule 2: d_model must be divisible by n_heads

```
✓  d_model=512, n_heads=8  → d_k = 512/8  = 64    (clean split)
✓  d_model=512, n_heads=4  → d_k = 512/4  = 128   (clean split)
✗  d_model=512, n_heads=7  → d_k = 512/7  = 73.1  (can't split evenly → crash)
```

Rule 3: Source and Target can have different vocab sizes, but d_model must be same

```
✓  src_vocab=10000 → embed to 512
   tgt_vocab=8000  → embed to 512     (different vocab, same d_model → works)

✗  src_embedding = 512
   tgt_embedding = 256                 (different d_model → cross-attention breaks)
```

Rule 4: Encoder output shape must match decoder cross-attention input

```
Encoder output (memory):  (batch, src_seq_len, 512)
                                                ↑
Decoder cross-attention expects K, V with:      512 dims

These MUST match — decoder uses encoder output as K and V in cross-attention.
```

Rule 5: Causal mask size must match target sequence length

```
Target has 5 tokens → mask must be (5, 5)
Target has 10 tokens → mask must be (10, 10)

Wrong size → masks wrong positions → model sees future words or crashes
```

Rule 6: Target input and target labels must be shifted by 1

```
Target input (decoder sees):   [, je, suis, un, étudiant]
Target labels (correct answer): [je, suis, un, étudiant, ]
                                  ↑ shifted by 1 position

Why? The model predicts the NEXT word at each position:
  Given               → predict "je"
  Given , je          → predict "suis"
  Given , je, suis    → predict "un"

If not shifted → model just copies input instead of learning to predict
```

Rule 7: Positional encoding dimension = d_model

```
embedding:           (4, 512)
positional_encoding: (4, 512)   ← must match for addition

final = embedding + positional_encoding

Different sizes → can't add → crash
```

Soft Rules (break these → model works but performs poorly)

Rule 8: Scale attention by √d_k

```
Not strictly required, but without it:
  Large dot products → softmax saturates → vanishing gradients → bad training

With scaling:    scores / √64 = reasonable range → smooth softmax → healthy gradients
Without scaling: scores are huge → softmax = [0.99, 0.00, 0.00, 0.01] → can't learn
```

Rule 9: Number of encoder layers ≈ number of decoder layers

```
Original paper: 6 encoder layers + 6 decoder layers (balanced)

You CAN use different numbers (e.g., 8 encoder + 4 decoder), but balanced is standard.
```

Rule 10: FFN inner dimension = 4 × d_model (convention)

```
d_model = 512  →  d_ff = 2048  (4 × 512)

This 4× ratio was found to work well in the original paper.
You can change it, but 4× is the proven default.
```

Quick Reference Table

#	Rule	What	Break it?
1	d_model same everywhere	512 in = 512 out at every layer	Crash
2	d_model ÷ n_heads = integer	512 ÷ 8 = 64 ✓	Crash
3	Encoder & Decoder same d_model	Both embed to 512	Crash
4	Encoder output matches decoder cross-attn	Both 512 dims	Crash
5	Mask size = target seq length	5 tokens → 5×5 mask	Wrong results
6	Target shifted by 1	Input: `<sos>` + tokens, Label: tokens + `<eos>`	Won't learn
7	Positional encoding dim = d_model	Both 512	Crash
8	Scale by √d_k	Divide attention scores	Bad training
9	Encoder layers ≈ Decoder layers	6 + 6 (balanced)	Suboptimal
10	d_ff = 4 × d_model	2048 = 4 × 512	Suboptimal

The one rule to remember: d_model is the backbone — it must be consistent from start to finish across the entire model.

---

7. Transformer Inputs, Parameters & Hyperparameters

What inputs does the Transformer need?

During Training

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          TRAINING INPUTS                                 │
├──────────────────────┼──────────────────────────────────────────────────┤
│ Input                │ Example                                          │
├──────────────────────┼──────────────────────────────────────────────────┤
│ src_batch            │ [[12, 45, 7, 892, 2, 0, 0],                      │
│ (source token IDs    │  [34, 56, 2, 0, 0, 0, 0]]                        │
│  padded to same len) │  shape: (batch_size, src_seq_len)                │
├──────────────────────┼──────────────────────────────────────────────────┤
│ tgt_input_batch      │ [[1, 15, 67, 23, 541, 0, 0],                     │
│ (target with     │  [1, 89, 2, 0, 0, 0, 0]]                        │
│  at the start)       │  shape: (batch_size, tgt_seq_len)                │
├──────────────────────┼──────────────────────────────────────────────────┤
│ tgt_target_batch     │ [[15, 67, 23, 541, 2, 0, 0],                     │
│ (target with     │  [89, 2, 0, 0, 0, 0, 0]]                        │
│  at the end — labels)│  shape: (batch_size, tgt_seq_len)                │
├──────────────────────┼──────────────────────────────────────────────────┤
│ tgt_mask             │ [[True, False, False, False],                    │
│ (causal mask to      │  [True, True,  False, False],                    │
│  block future words) │  [True, True,  True,  False],                    │
│                      │  [True, True,  True,  True ]]                     │
│                      │  shape: (tgt_seq_len, tgt_seq_len)               │
└──────────────────────┼──────────────────────────────────────────────────┘
```

```
Special token IDs:  0 = ,  1 = ,  2 = ,  3 =
```

During Inference

```
┌──────────────────────────────────────────────────────────────────────────┐
│                         INFERENCE INPUTS                                 │
├──────────────────────┼──────────────────────────────────────────────────┤
│ Input                │ Example                                          │
├──────────────────────┼──────────────────────────────────────────────────┤
│ src_tensor           │ [12, 45, 7, 892, 2]                              │
│ (source sentence     │  shape: (1, src_seq_len)                         │
│  with )          │ only ONE sentence, no padding needed             │
├──────────────────────┼──────────────────────────────────────────────────┤
│ generated tokens     │ starts as [1] ()                                 │
│ (grows each step)    │ → [1, 15] → [1, 15, 67] → ...                    │
│                      │  shape: (1, current_len) — grows each step       │
├──────────────────────┼──────────────────────────────────────────────────┤
│ tgt_mask             │ grows with each step:                            │
│ (causal mask, grows  │  step 1: (1,1)  step 2: (2,2)  step 3: (3,3)│
│  as output grows)    │                                                  │
└──────────────────────┼──────────────────────────────────────────────────┘
```

```
No tgt_target_batch needed — there's no correct answer to compare against.
The model generates freely.
```

Parameters (learned by the model during training)

These are the **weights** that the model learns. You don't set these — backpropagation updates them automatically.

Parameter	What it is	Shape	How it's learned
Source Embedding weights	Lookup table: token ID → vector	(src_vocab_size, d_model)	Learns which numbers best represent each source word
Target Embedding weights	Lookup table: token ID → vector	(tgt_vocab_size, d_model)	Learns which numbers best represent each target word
W_Q (Query weights)	Transforms input into Query vectors	(d_model, d_k) per head	Learns what each word should "ask for"
W_K (Key weights)	Transforms input into Key vectors	(d_model, d_k) per head	Learns what each word should "advertise"
W_V (Value weights)	Transforms input into Value vectors	(d_model, d_k) per head	Learns what information each word should "share"
W_O (Output projection)	Combines all heads back together	(d_model, d_model)	Learns how to mix information from all heads
FFN W1 (first layer)	Expands representation	(d_model, d_ff)	Learns complex patterns (expand step)
FFN W2 (second layer)	Compresses back	(d_ff, d_model)	Learns complex patterns (compress step)
FFN biases (b1, b2)	Offset values in FFN	(d_ff), (d_model)	Fine-tunes the FFN transformations
LayerNorm γ (gamma)	Scale factor for normalization	(d_model)	Learns optimal scale for each dimension
LayerNorm β (beta)	Shift factor for normalization	(d_model)	Learns optimal shift for each dimension
Output Head weights	Projects decoder output to vocab	(d_model, tgt_vocab_size)	Learns to pick the right word from vocabulary

Example — counting parameters:

```
With d_model=512, n_heads=8, d_ff=2048, n_layers=6, src_vocab=10000, tgt_vocab=8000:

Embeddings:
  Source embedding:     10000 × 512 = 5,120,000
  Target embedding:     8000 × 512  = 4,096,000

Per encoder layer (× 6 layers):
  Self-Attention:       4 × (512 × 512) = 1,048,576    (W_Q, W_K, W_V, W_O)
```

```
  FFN:                    512×2048 + 2048×512 = 2,097,152 (W1, W2)
  LayerNorm (×2):         2 × (512 + 512) = 2,048         (γ, β for each norm)
  Subtotal per layer:     ~3.1M
  All 6 layers:           ~18.9M

Per decoder layer (× 6 layers):
  Masked Self-Attention: 4 × (512 × 512) = 1,048,576
  Cross-Attention:       4 × (512 × 512) = 1,048,576
  FFN:                   2,097,152
  LayerNorm (×3):        3 × (512 + 512) = 3,072
  Subtotal per layer:    ~4.2M
  All 6 layers:          ~25.2M

Output Head:             512 × 8000 = 4,096,000

Total: ~57M parameters (all learned during training)
```

Hyperparameters (set by YOU before training)

These are the **choices you make** before training starts. The model doesn't learn these — you decide them.

Model Architecture Hyperparameters

Hyperparameter	What it controls	Typical value	How to think about it
`d_model`	Size of every vector in the model	512	Bigger = model understands more, but slower and needs more data. Must be same everywhere (Rule 1)
`n_heads`	Number of attention heads	8	More heads = captures more relationship types. Must divide d_model evenly (Rule 2)
`n_layers`	Number of encoder/decoder layers	6	More layers = deeper understanding, but slower and harder to train
`d_ff`	Inner size of feed-forward network	2048 (4×d_model)	Bigger = more capacity to learn complex patterns
`src_vocab_size`	Number of unique source words	10000-50000	Depends on your source language and tokenizer
`tgt_vocab_size`	Number of unique target words	10000-50000	Depends on your target language and tokenizer
`max_seq_len`	Maximum sentence length allowed	512	Longer = handles bigger sentences, but uses more memory
`dropout`	Randomly turns off neurons during training	0.1 (10%)	Prevents overfitting — forces model to not rely on any single neuron

Training Hyperparameters

Hyperparameter	What it controls	Typical value	How to think about it
`learning_rate (lr)`	How big each weight update step is	1e-4 (0.0001)	Too high = model overshoots and diverges. Too low = learns too slowly
`batch_size`	Number of sentence pairs processed together	32-128	Bigger = more stable gradients but needs more GPU memory
`n_epochs`	How many times to loop through all training data	10-100+	More = better learning, but risk overfitting (memorizing instead of learning)
`betas`	Momentum settings for Adam optimizer	(0.9, 0.98)	Controls how much past gradients influence current update. 0.98 is tuned for transformers
`eps`	Small number to prevent division by zero in Adam	1e-9	Just a safety value, rarely needs changing
`max_norm`	Gradient clipping threshold	1.0	Caps gradient size to prevent exploding gradients
`label_smoothing`	Softens the target distribution	0.1	0.0 = hard targets (100% on correct word). 0.1 = soft targets (90% correct, 10% spread)
`warmup_steps`	Gradually increase learning rate at start	4000	Prevents large unstable updates at the beginning when weights are random

How these hyperparameters affect the model:

```
Small model (fast, less accurate):
  d_model=128, n_heads=4, n_layers=2, d_ff=512
  Parameters: ~2M
  Good for: small datasets, quick experiments, learning

Medium model (balanced):
  d_model=512, n_heads=8, n_layers=6, d_ff=2048
  Parameters: ~57M
  Good for: real translation tasks, moderate datasets

Large model (slow, more accurate):
  d_model=1024, n_heads=16, n_layers=12, d_ff=4096
  Parameters: ~300M+
  Good for: large datasets, production systems, state-of-the-art results
```

Parameters vs Hyperparameters — the difference:

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│  Hyperparameters (YOU decide)        Parameters (MODEL learns)  │
│  ────────────────────────            ────────────────────────   │
│  d_model = 512                       W_Q = [[0.2, 0.8, ...],    │
│  n_heads = 8                                [0.1, 0.3, ...]]    │
│  n_layers = 6                        W_K = [[...], [...]]       │
│  learning_rate = 0.0001              W_V = [[...], [...]]       │
│  dropout = 0.1                       Embeddings = [[...], ...]  │
```

```
│  batch_size = 32                     FFN weights = [[...], ...]  │
│                                                                  │
│  Set BEFORE training                 Updated DURING training     │
│  Fixed throughout training           Change every epoch           │
│  Chosen by human (or tuning)         Learned by backpropagation  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```